Array dinamici

Description	Malloc(), Realloc(), Valgrind, Free(), differenza tra array statici & Dinamici

```
./ cerco nella directory corrente il file
   senza cerca il file da aprire in tutto il path
```

```
int *a; // dichiaro che sarà usata come array
a = malloc(n*sizeof(int));
if (a==NULL) { //faccio un semplice controllo per verificare che l'array non si null
  puts("Malloc fallita");
exit(3);
}

for (int i=0; i<n; i++) {
  inserisco n elementi
}
```

```
#include <malloc.h>
malloc(); Funzione utilizzata per gestire la memoria dinamica, ritorna un
          puntatore a carattere, corrispondente al punto di inizio, in memoria,
          della porzione riservata della dimensione "intera" passata come
          argomento; se la memoria richiesta non può essere allocata,
          ritorna un puntatore nullo.
```

```
sizeof(); restituisce la dimensione del tipo di dato da allocare e specifica il
          numero esatto di byte
```

❓ **Differenza tra Array statici e dinamici**

- **Array Statici:** Gli **array** la cui dimensione viene specificata all'interno del codice si chiamano **array** statici
- **Array Dinamici:** Hanno una dimensione variabile

`Realloc()`

```
realloc(memoria vecchia, memoria nuova);
Funzione che modifica le aree precedentemente allocate anche in una fase
successiva, ad esempio ampliando la memoria. Può essere utilizzando solo se
precedentemente l'avevo preso con una malloc()
```

`Valgrid`

```
valgrind --tool = toolname program
Sostituire toolname con il nome dello strumento desiderato
```

```
Sostituire program con il programma desiderato da usare con Valgrind
```

Valgrind è un framework in grado di fornire una strumentazione ai binari dello spazio utente.

È presente con un numero di strumenti utilizzabili per il tracciamento e l'analisi delle prestazioni dei programmi.
Avverte dell'uso di memoria non inizializzata e una assegnazione incorretta o una deallocazione della memoria stessa.

```
valgrind program

Rileva e riporta un certo numero di errori della memoria difficili da
rilevare e diagnosticare, come ad esempio un accesso della memoria non previsto,
l'uso di valori non inizializzati o non definiti, memoria heap resa disponibile
non correttamente, puntatori sovrapposti e perdite di memoria.
```

> ⚠️ *ritorna errori specifici in base al tipo di problema rilevato ma non impedisce la loro generazione.*

`Free()`

Serve a rilasciare la memoria precedentemente allocata in maniera dinamica con la funzioni `malloc()` . Dopo che un puntatore è stato passato come argomento alla funzione free, diventa un **dangling pointer**, cioè un *puntatore pendente*. Infatti esso fa riferimento a un'area di memoria non più disponibile.

```
void free( void* ptr );
```

quando la memoria non si usa più si libera, ad esempio poco prima di finire il programma

```
.
.
.
 free();

 return 0;
}
```

Con `valgrid` posso controllare anche se ho liberato tutta la memoria

```
echo $?
 2

con echo vedo l'exit che ha effettuato il programma
```

Creazione array dinamico di n elementi

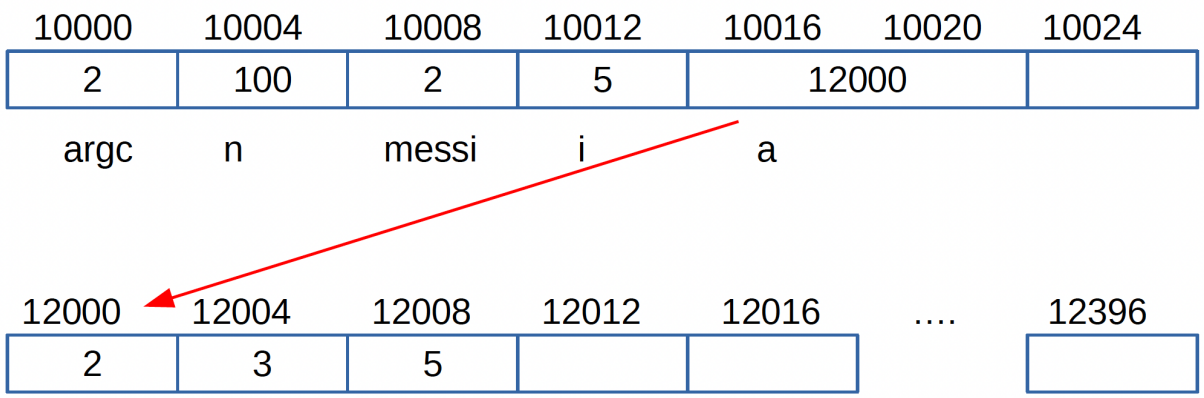
```
// crea un array dinamico inizialmente di 10 elementi
int *a;  // dichiaro che a sarà usata come array
int size = 10; // dimensione attuale dell'array
int messi = 0; // numero di elementi attualmente dentro l'array

a = malloc(n * sizeof(int)); // aumento la dimensione dell'array
```

```
A[7] = 23

A[7] => 12.000 + 7 * sizeof(int)
  12.000 = Indirizzo di memoria dove sto inserendo i dati dell'array
  7In quale indice dell'array dobbiamo inserire l'elementosizeof()restituisce la dimensione del tipo di dato che dobbiamo inserire

sizeof(int) => 4 byte
```



📌 ***Aumentare la dimensione*** dell'array può essere **costoso** e quindi si fa **raramente**

💡 La malloc restituisce il puntatore alla cella di memoria dove stiamo allocando l'array, con `valgrid` possiamo vedere l'indirizzo di memoria

```
if(messi==size) {
  size = 2*size; // se la tabella è piena raddoppio la dimensione
  a = realloc(a,size*sizeof(int)); // aggiorno la dimensione della memoria
  if(a==NULL) termina("Realloc fallita");
}
```

💡 realloc(*memoria vecchia*, *memoria nuova*);

Con `realloc()` posso anche diminuire la memoria dell'array, la funzione copia i primi n elementi che noi vogliamo nell nuova memoria e il resto li elimina. Come uso l'array non cambia

Con la `free()` libero solo il contenuto dell'array ma non perdo la cella di memoria dov'è memorizzato l'array